

08 — Best Practices

[← Powrót do README \(./README.md\)](#) | [Następna: Metryki Jakości → \(./09-quality-metrics.md\)](#)

Code Quality Standards

Python Style

Narzędzie	Rola	Konfiguracja
Black	Auto-formatter	<code>line-length = 100</code>
Ruff	Linters (zastępuje flake8 + isort + pyupgrade)	<code>select = ["E", "F", "I", "UP", "B", "SIM"]</code>
mypy	Static type checker	<code>strict = true</code>
radon	Complexity analysis	<code>cc -s -a --total-average</code>

Naming Conventions

```
# Moduły: snake_case
feed_adapter.py
ingestion_pipeline.py

# Klasy: PascalCase
class FeedAdapterRegistry:
class CanonicalIIOC:

# Funkcje/metody: snake_case
def validate_config(self, config: FeedConfig) -> ConfigValidationResult:

# Stałe: UPPER_SNAKE_CASE
MAX_FETCH_LIMIT = 10000
HIGH_CONFIDENCE_THRESHOLD = 70
DEFAULT_CIRCUIT_COOLDOWN_S = 300

# Private: _prefix
def _map_to_canonical(self, raw_item: Dict) -> Optional[CanonicalIIOC]:
```

Type Hints

```
# ✓ POPRAWNIE: pełne type hints
def search_indicators(
    query: str | None = None,
    source: str | None = None,
    limit: int = 100,
    offset: int = 0,
) -> tuple[list[Indicator], int]:
    ...

# ✗ NIEPOPRAWNIE: brak type hints
def search_indicators(query=None, source=None, limit=100, offset=0):
    ...
```

Docstrings (Google Style)

```
def fetch(
    self,
    config: FeedConfig,
    context: FetchContext,
) -> FetchResult:
    """Pobierz dane z external API i mapuj na CanonicalIOC.

    Args:
        config: Konfiguracja feeda (auth, URL, filters).
        context: Runtime context (since, limit, correlation_id).

    Returns:
        FetchResult z listą CanonicalIOC i statystykami.

    Raises:
        ConnectionError: Jeśli external API jest nieosiągalne
            po wyczerpaniu retry.

    Example:
        >>> adapter = CrowdSecAdapter()
        >>> result = adapter.fetch(config, context)
        >>> print(f"Fetched {result.stats.total_fetched} items")
    """
```

Git Workflow

Trunk-Based Development (rekomendowany)

```
gitgraph
  commit id: "main"
  branch feature/auth
  commit id: "add login"
  commit id: "add RBAC"
  checkout main
  merge feature/auth id: "PR #42"
  branch feature/adapters
  commit id: "contracts"
  commit id: "registry"
  checkout main
  merge feature/adapters id: "PR #45"
  commit id: "v1.5.0 tag"
```

Zasady:

1. `main` zawsze deployable (green CI)
2. Feature branches: `feature/<ticket>-<opis>` (max 2-3 dni)
3. Pull Request z min. 1 approval
4. Squash merge (clean history)
5. Delete branch po merge

Branch Naming

```
feature/I0C-142-admin-authentication
feature/I0C-161-crowdsec-adapter
bugfix/I0C-200-export-timeout
hotfix/I0C-201-csrf-bypass
chore/I0C-210-update-dependencies
```

Commit Message Conventions (Conventional Commits)

```
<type>(<scope>): <description>

[optional body]

[optional footer(s)]
```

Typy

Type	Opis	Przykład
feat	Nowa funkcjonalność	feat(auth): add login endpoint with rate limiting
fix	Bug fix	fix(adapter): handle empty response from CrowdSec API
refactor	Refactoring (bez zmiany zachowania)	refactor(main): extract indicator service
test	Dodanie/zmiana testów	test(adapter): add contract tests for MISP adapter
docs	Dokumentacja	docs(api): add OpenAPI specification
chore	Maintenance	chore(deps): update SQLAlchemy to 2.0.37
security	Security fix	security(auth): patch CSRF token validation
perf	Performance	perf(export): add streaming for large exports

Code Review Guidelines

Checklist dla reviewera

Funkcjonalność:

- [] Czy kod robi to, co opisuje PR?
- [] Czy acceptance criteria są spełnione?
- [] Czy edge cases są obsługiwane?

Jakość:

- [] Czy są type hints na public API?
- [] Czy są docstrings na klasach/metodach?
- [] Czy nie ma duplikacji kodu?
- [] Czy complexity jest akceptowalna (<15 CC)?

Bezpieczeństwo:

- [] Czy input jest walidowany?
- [] Czy nie ma SQL injection risk?
- [] Czy secrets nie są logowane?
- [] Czy CSRF jest obsługiwane (state-changing endpoints)?

Error Handling Patterns

```
# 1. Custom exception hierarchy
class IOServiceError(Exception):
    """Base exception."""
    def __init__(self, message: str, code: str = "INTERNAL_ERROR"):
        self.message = message
        self.code = code

class ValidationError(IOServiceError):
    def __init__(self, message: str, field: str | None = None):
        super().__init__(message, code="VALIDATION_ERROR")
        self.field = field

class NotFoundError(IOServiceError):
    def __init__(self, entity: str, id: Any):
        super().__init__(f"{entity} not found: {id}", code="NOT_FOUND")

class AuthenticationError(IOServiceError):
    def __init__(self, message: str = "Authentication required"):
        super().__init__(message, code="AUTHENTICATION_REQUIRED")

# 2. Flask error handler
@app.errorhandler(IOServiceError)
def handle_ioc_error(error):
    return jsonify({
        "error": {
            "code": error.code,
            "message": error.message,
        }
    }), _status_code_for(error)

# 3. Nigdy nie łykaj wyjątków cicho
# ❌ NIEPOPRAWNIE:
try:
    result = adapter.fetch(config, context)
except Exception:
    pass # Cichy błąd!

# ✅ POPRAWNIE:
try:
    result = adapter.fetch(config, context)
except Exception as e:
    logger.error("fetch_failed", extra={"source": source_id, "error": str(e)})
    circuit_breaker.record_failure(source_id)
    raise
```



Logging Standards

Structured Logging (JSON)

```
# ✅ POPRAWNIE: structured z extra dict
logger.info(
    "feed_sync_complete",
    extra={
        "source": "crowdsec",
        "fetched": 1234,
        "duration_ms": 456,
        "correlation_id": "req_abc123",
    },
)

# ❌ NIEPOPRAWNIE: f-string (nieparsowalny)
logger.info(f"CrowdSec sync done: fetched {count} in {duration}ms")
```

Log Levels

Level	Kiedy używać	Przykład
ERROR	Błąd wymagający uwagi	Fetch failed, DB connection lost
WARNING	Anomalia, ale system działa	Circuit open, rate limited
INFO	Ważne zdarzenia biznesowe	Feed sync complete, user login
DEBUG	Szczegóły diagnostyczne	Raw API response, mapping details



Performance Optimization Guidelines

- Database:** Używaj indeksów, EXPLAIN ANALYZE, unikaj N+1
- Caching:** Redis TTL dla exportów (300s), health (5s)
- Pagination:** Zawsze server-side, max 100 items/page
- Lazy loading:** Nie ładuj relacji ORM jeśli nie potrzebne
- Batch operations:** Bulk INSERT/UPDATE zamiast row-by-row
- Async exports:** Duże eksporty (≥10k) przez async job queue
- Connection pooling:** SQLAlchemy pool_size=10, max_overflow=20
- Profiling:** cProfile / py-spy dla hot path identification



Dependency Management

- Pin major+minor:** `Flask>=3.1,<4` (nie `Flask>=3`)

2. **Weekly updates:** Dependabot / Renovate
 3. **Security patches:** $\leq 4h$ dla critical, $\leq 7d$ dla high
 4. **Lock file:** `pip-compile` lub `pip freeze > requirements.lock`
 5. **Audit:** `safety check` w CI pipeline
 6. **License check:** Verify compatibility before adding dependency
-

[← Powrót do README](#) (./README.md) | [Następna: Metryki Jakości](#) → (./09-quality-metrics.md)